Diseño y Arquitectura de Software

Módulo 6. Contenerización en aplicaciones escalables

Facultad de
Ingeniería

VIGILADA MINEDUCACIÓN

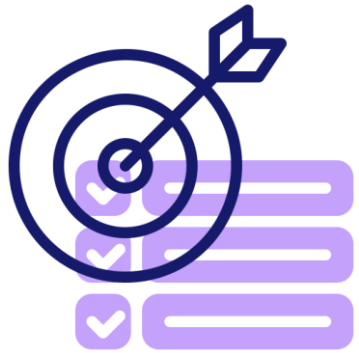


Universidad de La Sabana

Universidad de La Sabana | Facultad de **Ingeniería**



Bienvenidos

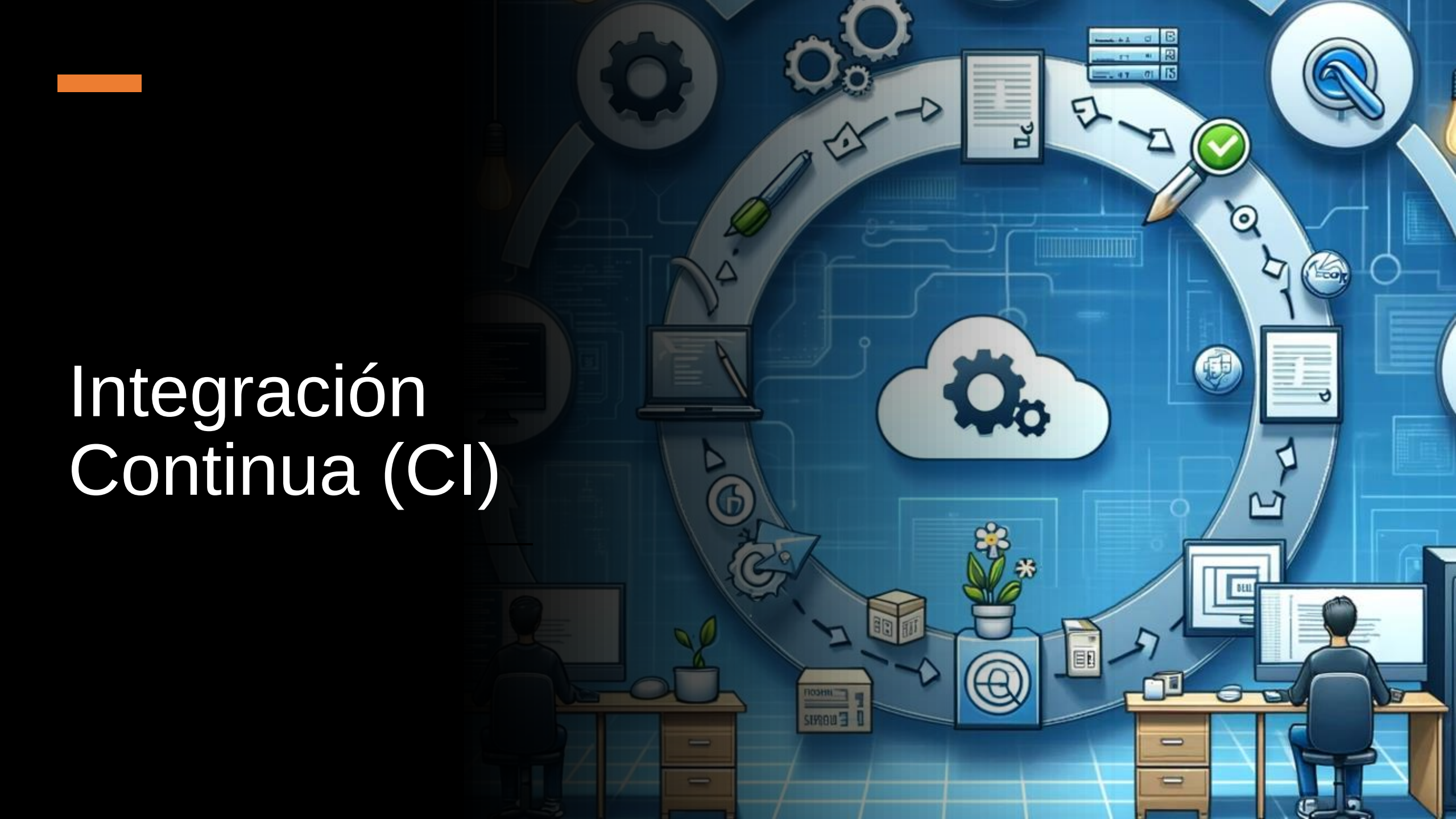


En este módulo 6, exploraremos cómo las tecnologías de contenerización y orquestación pueden transformar la manera en que desarrollamos, desplegamos y gestionamos nuestras aplicaciones. Nos enfocaremos en:

- Integración Continua (CI): Automatización de la integración de cambios de código.
- Docker: Creación y gestión de contenedores.
- Kubernetes: Orquestación de aplicaciones contenerizadas.
- Despliegue Continuo (CD): Automatización del despliegue de nuevas versiones de software.

A través de ejercicios prácticos, aplicaremos estos conceptos en escenarios reales, proporcionándoles las habilidades necesarias para implementar y gestionar aplicaciones escalables.

Integración Continua (CI)

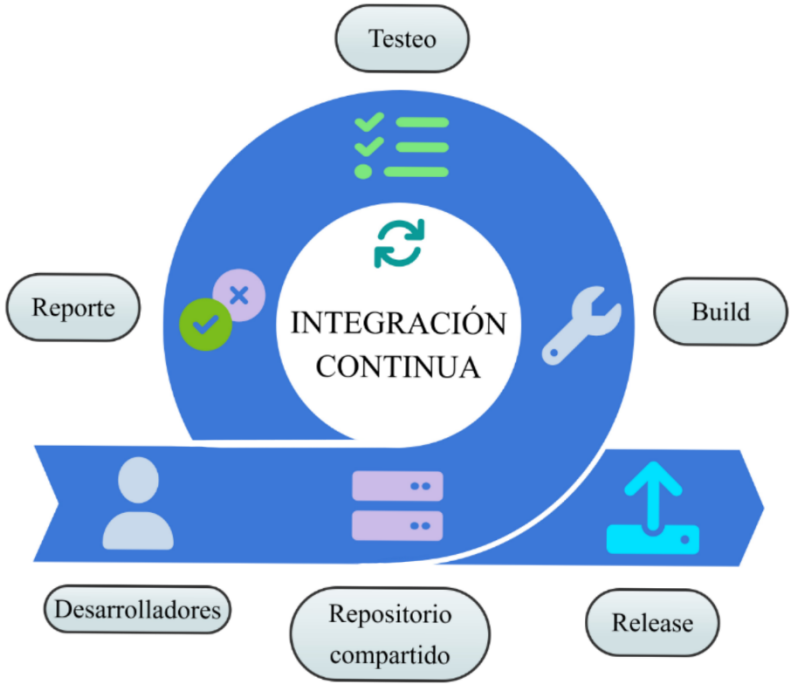




¿Qué es la Integración Continua (CI)?

La **Integración Continua (CI)** es una práctica de desarrollo de software donde los desarrolladores integran cambios en el código frecuentemente.

- Práctica de integrar cambios frecuentemente
- Verificación automática de integraciones



Beneficios de CI

Detección temprana de errores

Automatización



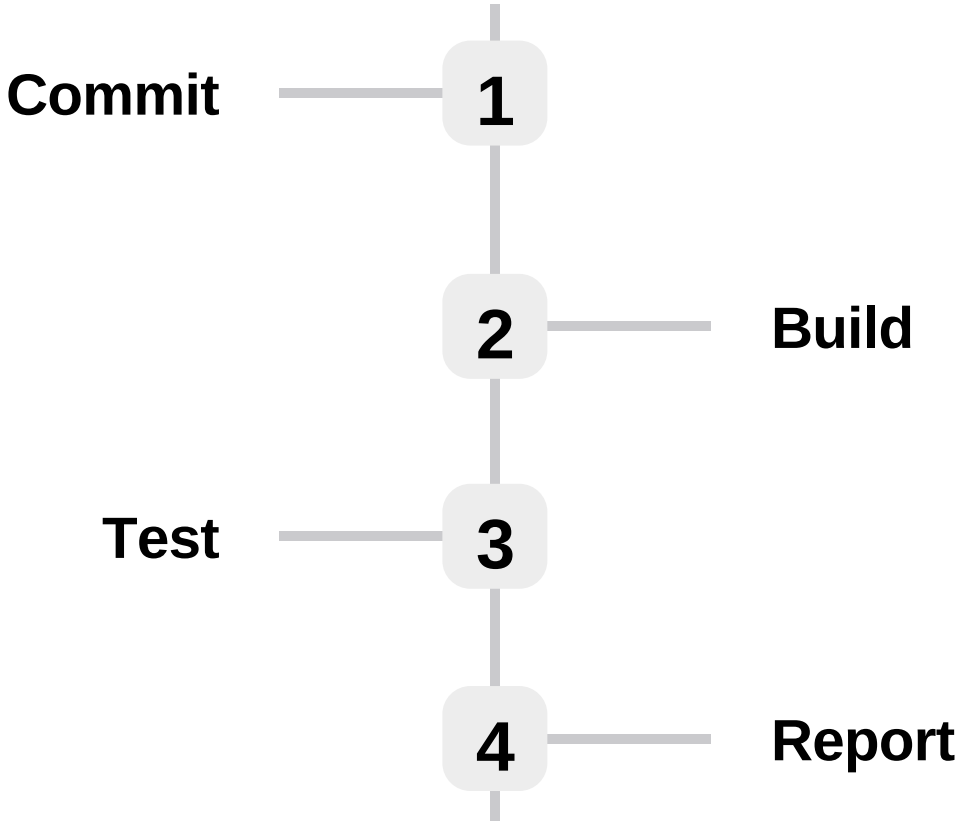
Mejora de la calidad del software

Colaboración efectiva





Flujo de Trabajo de CI





Herramientas Populares de CI

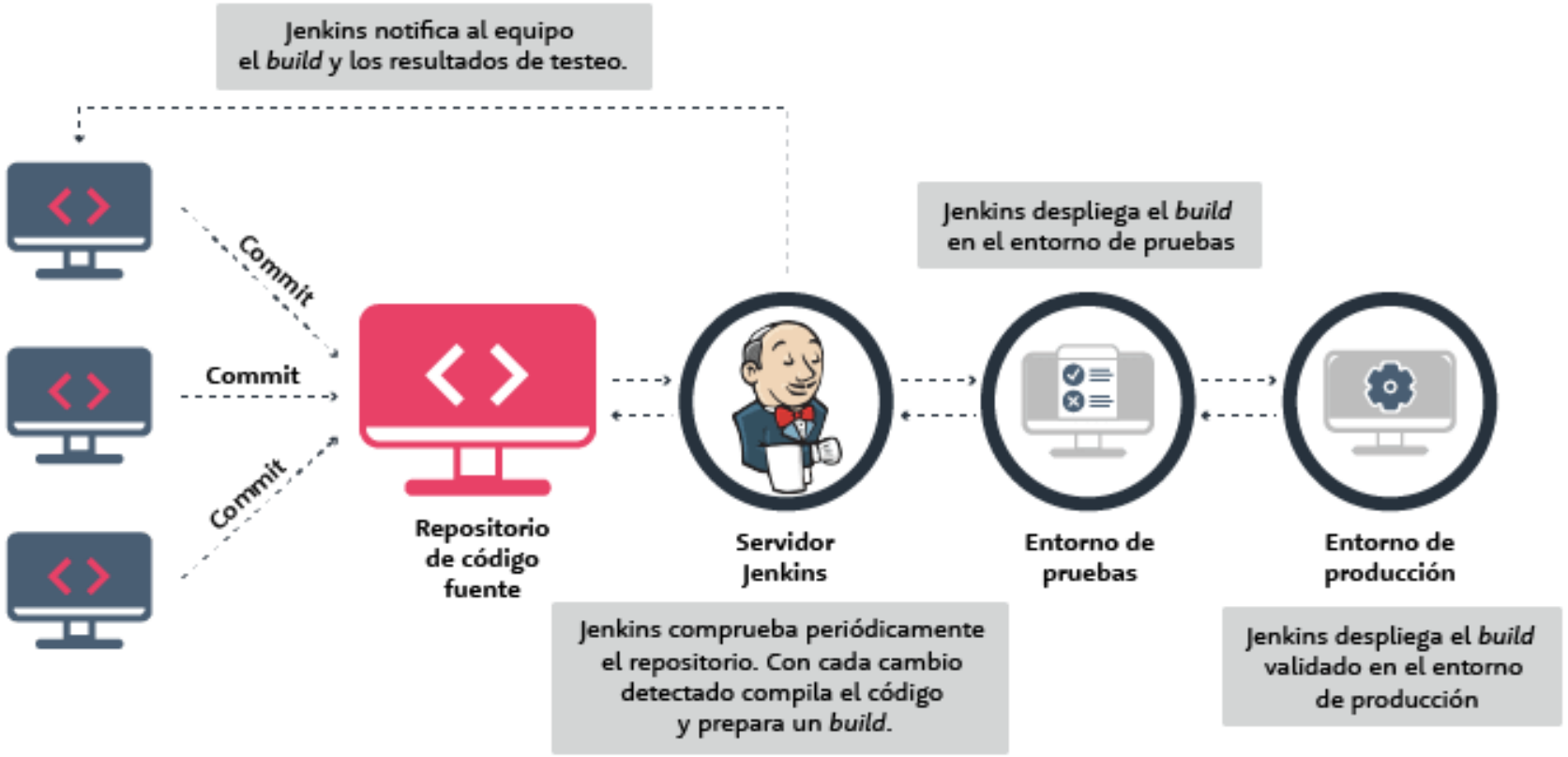
Jenkins

GitLab

Travis CI

circleci

Jenkins


Jenkins notifica al equipo el *build* y los resultados de testeo.
Commit
Commit
Commit
Repositorio de código fuente
Jenkins despliega el *build* en el entorno de pruebas
Servidor Jenkins
Entorno de pruebas
Entorno de producción
Jenkins comprueba periódicamente el repositorio. Con cada cambio detectado compila el código y prepara un *build*.
Jenkins despliega el *build* validado en el entorno de producción


Lobby Ad Portas



Estructura de Organización en Jenkins

Jenkins Master

Responsabilidades:

- Gestiona el entorno de construcción y distribución de tareas.
- Administra la interfaz de usuario web.
- Programa las tareas de construcción.
- Monitorea las ejecuciones de las tareas y reporta los resultados.
- Distribuye las cargas de trabajo a los nodos esclavos.

Jenkins Slave

Responsabilidades:

- Ejecuta las tareas de construcción distribuidas por el servidor maestro.
- Puede estar configurado en diferentes entornos para pruebas específicas.
- Permite el escalado horizontal al distribuir la carga de trabajo entre múltiples nodos.



Estructura de Organización en Jenkins

Flujo de Trabajo

1. **Programación de Tareas:** El servidor maestro programa las tareas de construcción y determina qué nodo esclavo las ejecutará.
2. **Distribución de Tareas:** El servidor maestro envía las tareas a los nodos esclavos adecuados.
3. **Ejecución de Tareas:** Los nodos esclavos ejecutan las tareas de construcción y pruebas.
4. **Reporte de Resultados:** Los nodos esclavos reportan los resultados de vuelta al servidor maestro.
5. **Visualización:** El servidor maestro presenta los resultados a través de la interfaz de usuario web.

Jenkins: Un Ejemplo de CI

Instalación fácil



Configuración flexible

Amplia gama de plugins





Ejemplo Práctico con GitLab CI - Paso a Paso

GitLab

1. Crear proyecto en GitLab
2. Agregar .gitlab-ci.yml
3. Commit y push
4. Verificar ejecución del pipeline



Mejores Prácticas en CI

- Commits frecuentes
- Pruebas automatizadas
- Revisión de código
- Monitoreo continuo



Contenedores

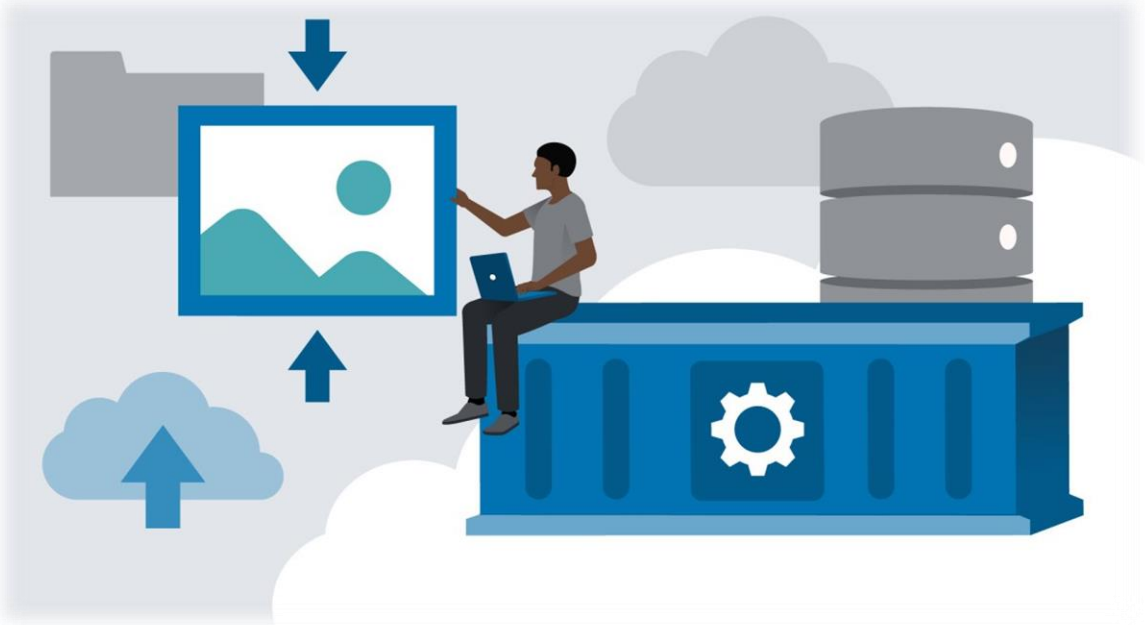


¿Qué es un Contenedor?

La **contenerización** es una técnica que se utiliza en diferentes contextos, y su significado puede variar según el ámbito en el que se aplique. A continuación, te proporciono algunas definiciones y explicaciones:

la **contenerización** se refiere al uso de **contenedores** para empaquetar aplicaciones y sus dependencias en un entorno portátil definido por software.

Estos contenedores son espacios de usuario aislados que comparten el mismo sistema operativo subyacente.





Ventajas

Eficiencia

La contenerización es más eficiente que la virtualización tradicional, ya que permite dividir los sistemas operativos en fragmentos que se pueden utilizar de manera más flexible.

Portabilidad

Los contenedores permiten a los desarrolladores escribir el código de las aplicaciones una vez y ejecutarlo en cualquier lugar.

Aislamiento

Cada contenedor proporciona un espacio aislado para la aplicación, lo que mejora la seguridad y facilita la administración.



¿Qué es Docker?

Docker es una plataforma de contenedores de software que permite empaquetar y ejecutar aplicaciones en diferentes entornos de una manera sencilla y escalable.

1 Contenedores

Una forma de virtualización a nivel de sistema operativo

2 Imágenes

Archivos de solo lectura que contienen el código, las librerías y las dependencias de una aplicación

3 Escalabilidad

Escalar aplicaciones y servicios de manera sencilla y eficiente



Ventajas

Entornos reproducibles

Asegura que las aplicaciones se ejecuten de la misma manera en diferentes entornos

Agilidad y eficiencia

Permite un desarrollo y despliegue más rápido y automatizado

Portabilidad

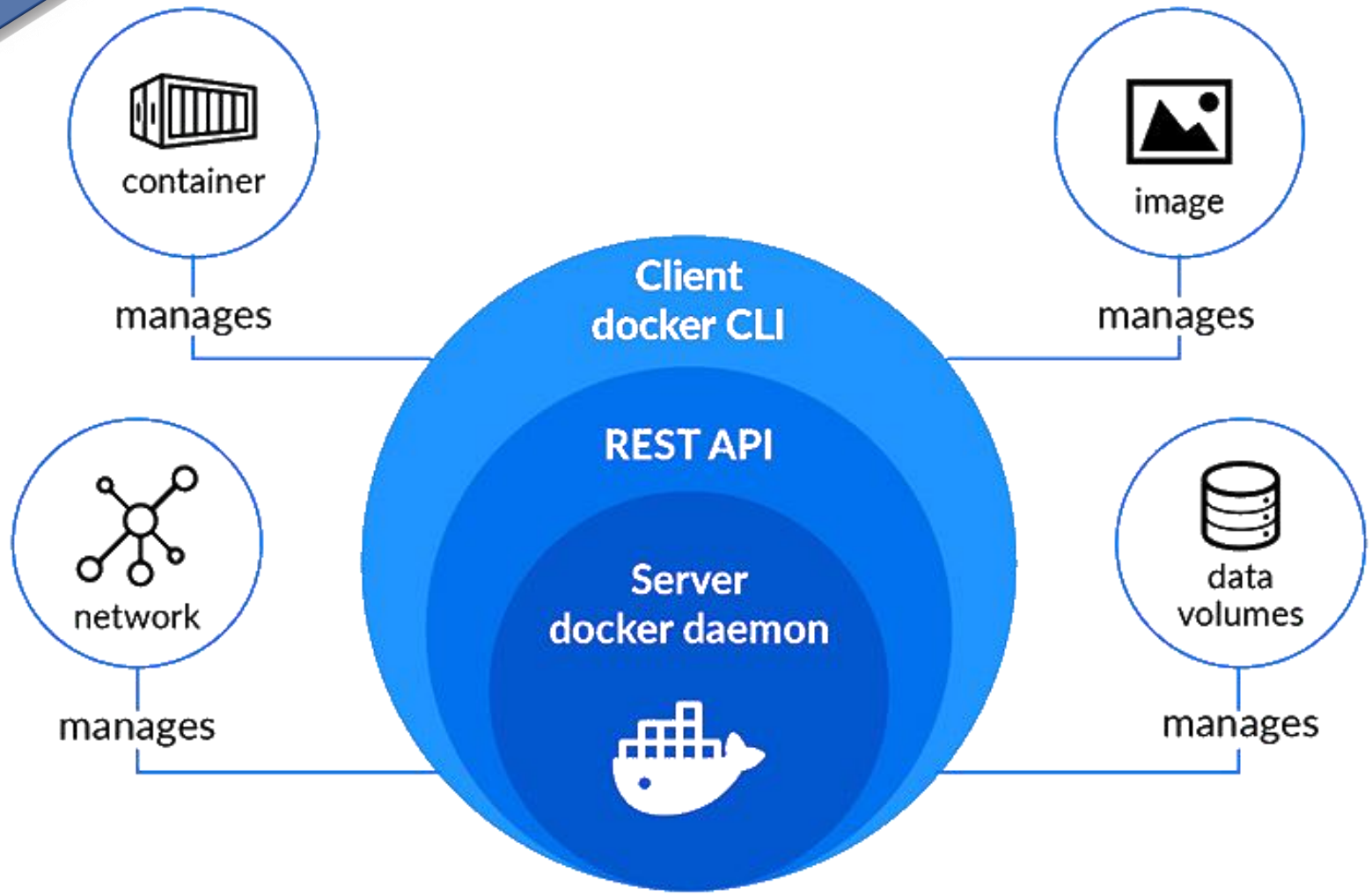
Las aplicaciones empaquetadas en contenedores pueden ejecutarse en cualquier plataforma



Componentes de Docker

- Docker Engine
- Docker Hub
- Docker Compose
- Docker Swarm

Docker Engine

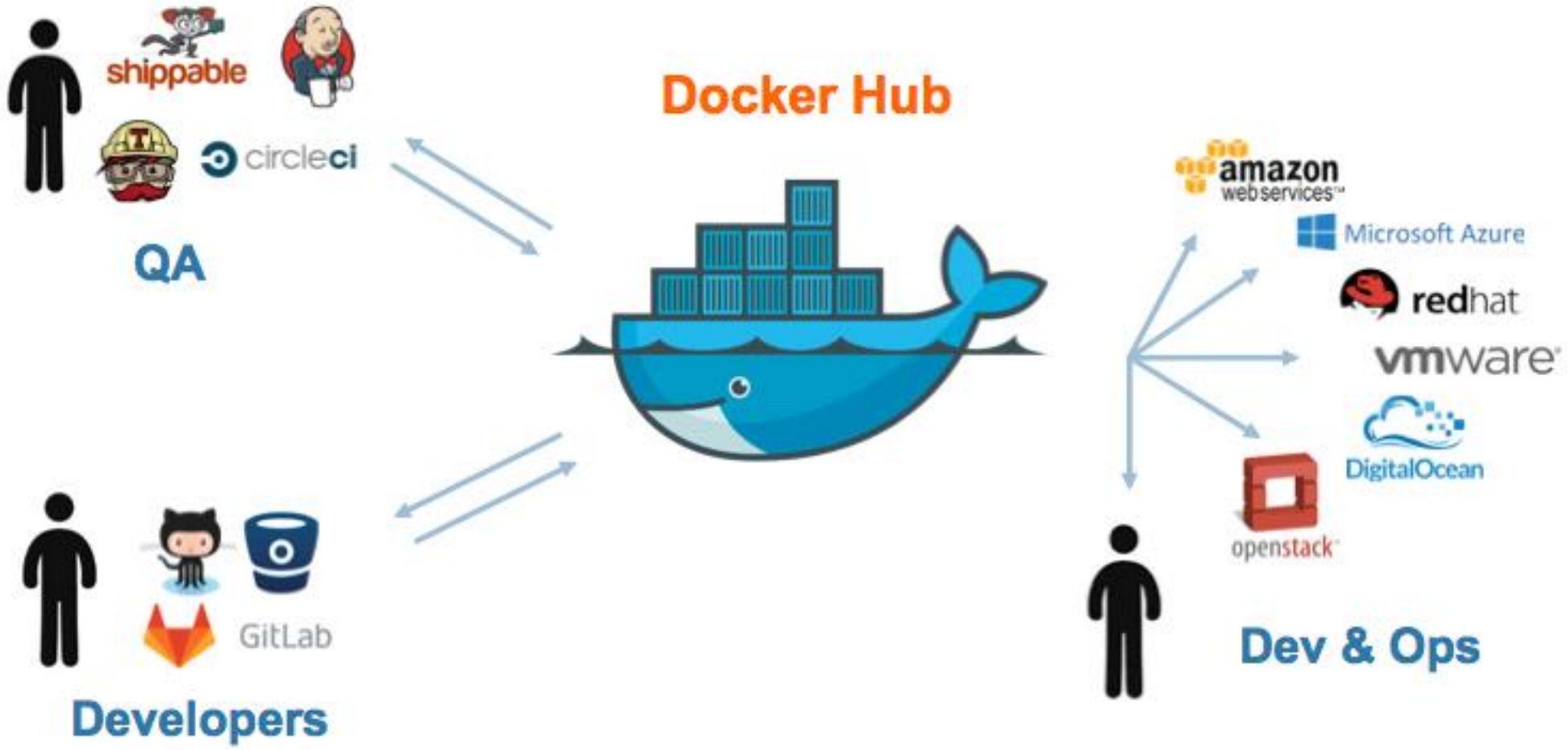




Facultad de
Ingeniería

Universidad de
La Sabana

Docker Hub

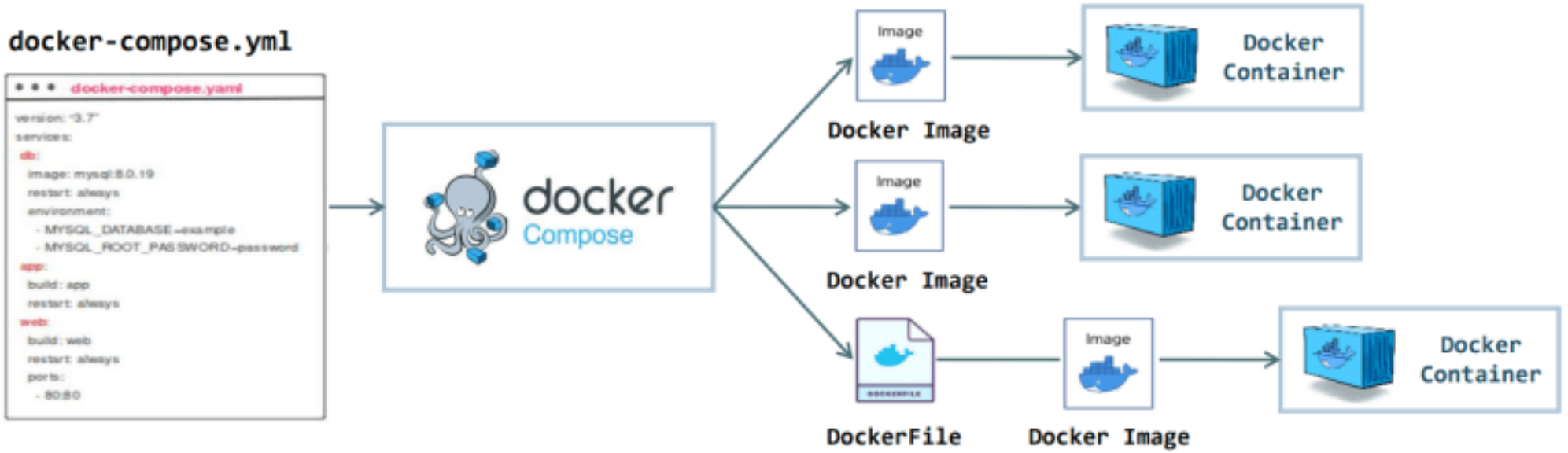


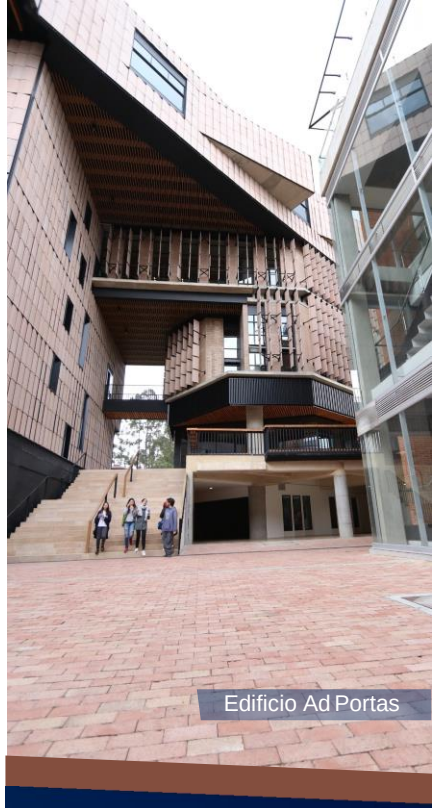


Facultad de
Ingeniería

Universidad de
La Sabana

Docker Compose

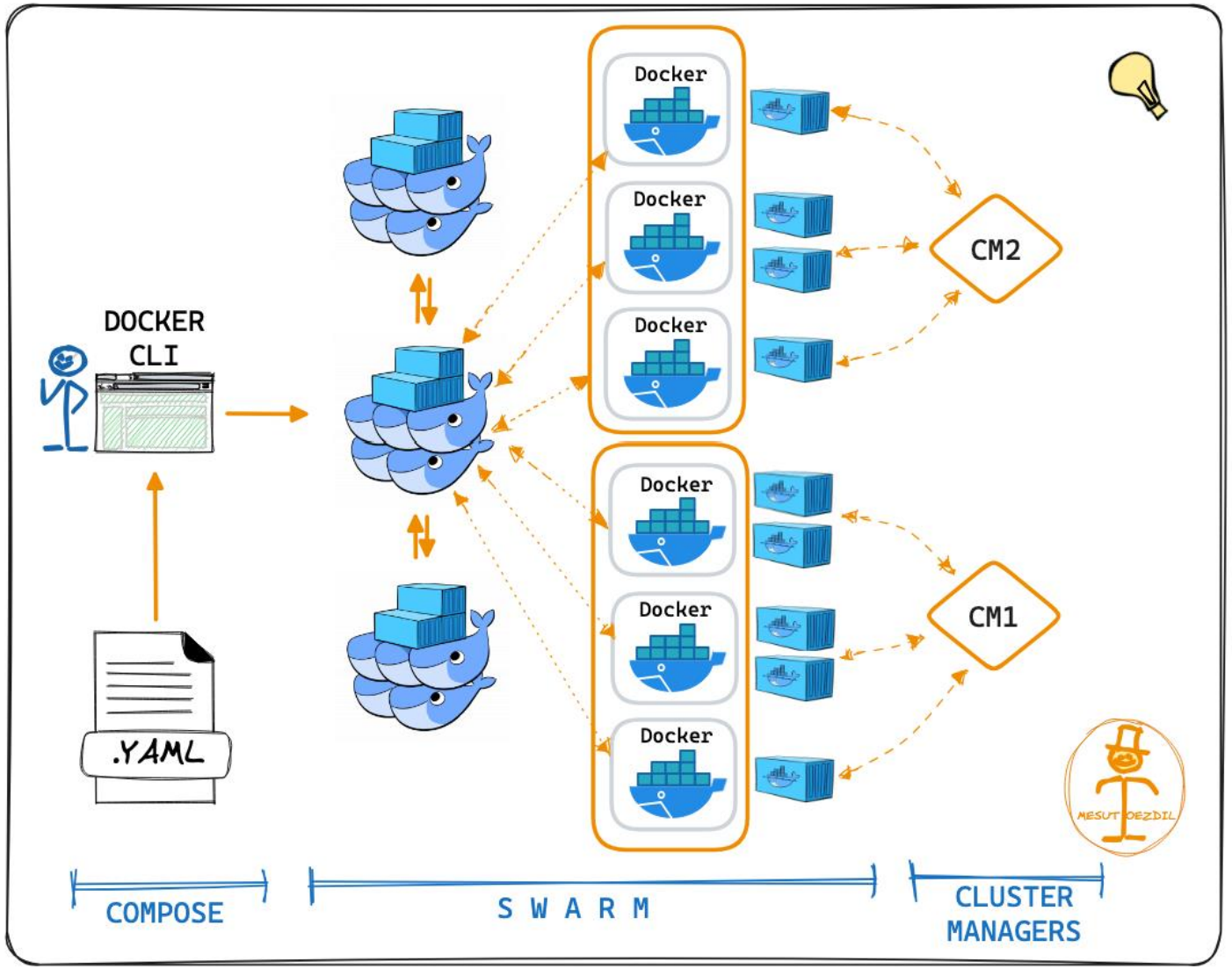




Facultad de
Ingeniería

Universidad de
La Sabana

Docker Swarm





Facultad de
Ingeniería

Universidad de
La Sabana



Ejemplos de uso y casos de uso comunes

Entornos de desarrollo y pruebas

Crear entornos aislados y reproducibles

Despliegue en la nube

Empaquetar y distribuir aplicaciones fácilmente

Arquitectura de microservicios

Construir aplicaciones modulares y escalables

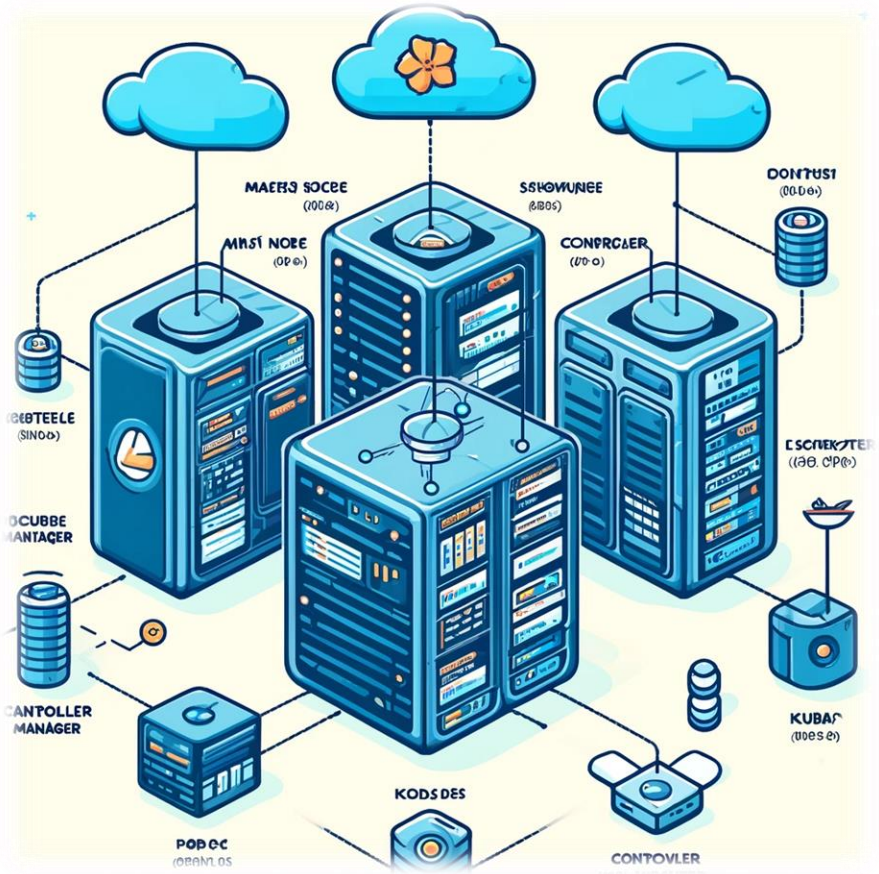
Servicios de base de datos

Ejecutar y escalar bases de datos en contenedores



¿Qué es Kubernetes?

Kubernetes es una **plataforma de orquestación de contenedores** que automatiza el despliegue, escalado y gestión de aplicaciones en contenedores.

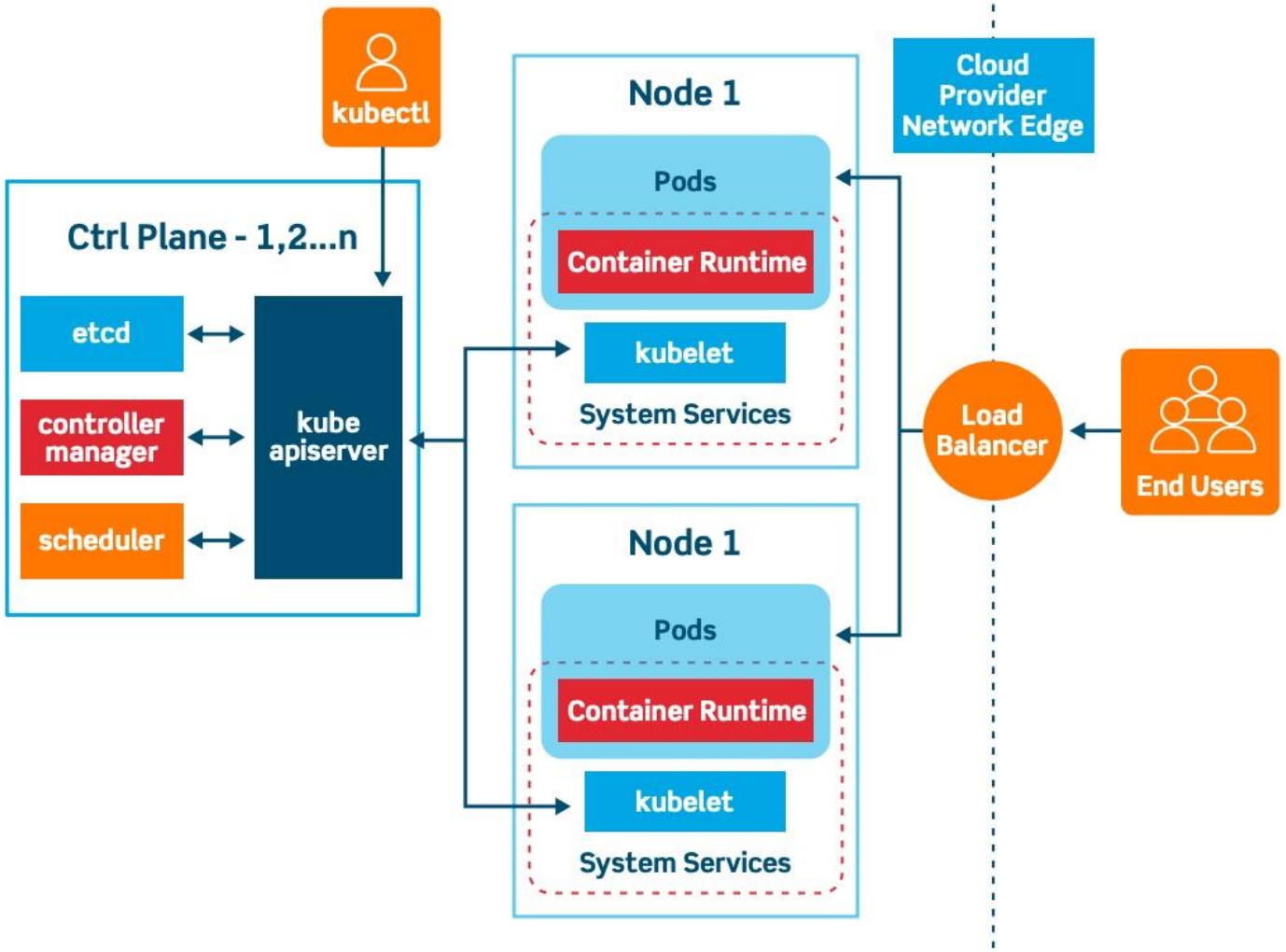


Master Node: Control Plane (API Server, Scheduler, Controller Manager, etcd).

Worker Nodes: Nodo que ejecuta los contenedores (Kubelet, Kube-proxy, Container Runtime).



Arquitectura de Kubernetes





Instalación de Kubernetes

Minikube

Desarrollo local

kubeadm

Producción

Servicios en la nube

Amazon Web Services (EKS)
Azure (AKS)
Google Cloud (GKE)



Despliegue de Aplicaciones

1

Crear Pods

Contiene los contenedores

2

Deployment

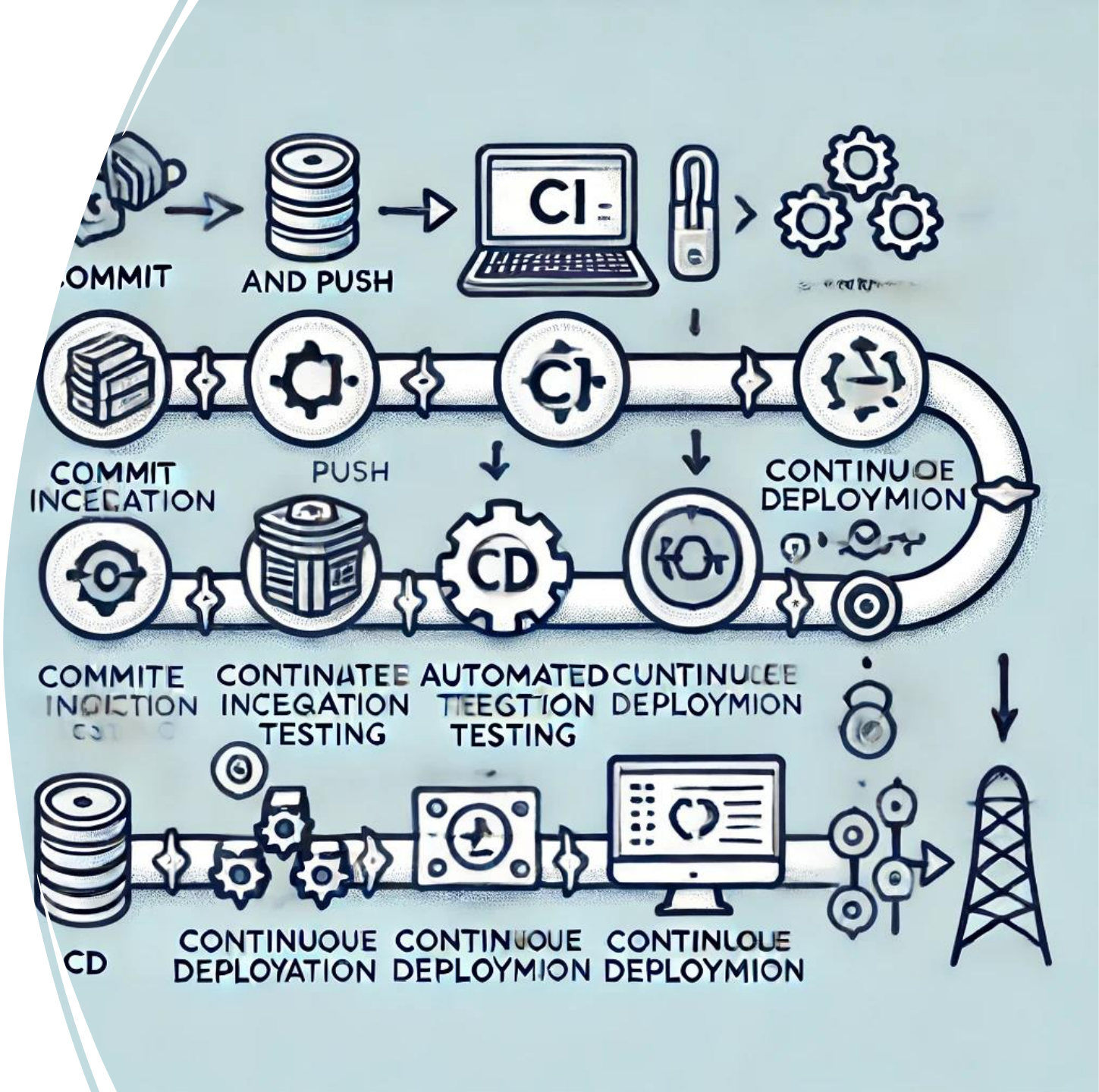
Gestiona Pods

3

Service

Exposicion del pod

Despliegue continuo (CD)



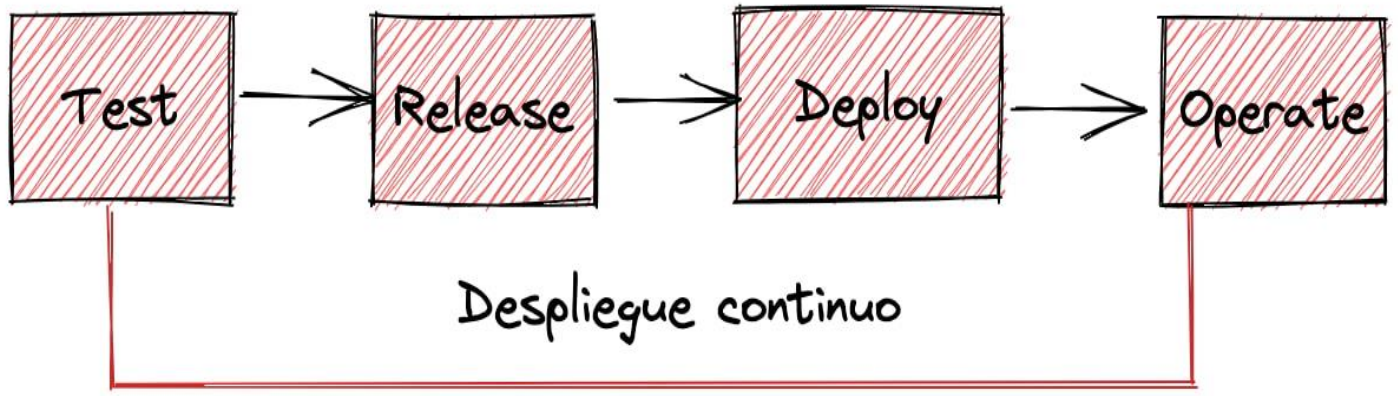


¿Qué es Despliegue Continuo (CD)?

Práctica de desarrollo de software en la que todas las modificaciones que pasan las pruebas automatizadas se despliegan automáticamente en producción.

Propósito

- Asegurar que el software esté siempre en un estado desplegable.
- Facilitar la entrega rápida y eficiente de nuevas versiones de la aplicación.



Beneficios de CD

Entrega Rápida de Funcionalidades



Mejora de la Productividad del Equipo

Mayor Calidad del Software

Mayor Confianza en los Despliegues





Flujo de Trabajo de CI

1. Commit y Push
2. Integración Continua (CI)
3. Despliegue Automático
4. Monitoreo y Feedback



Herramientas Populares de CD

Jenkins

GitLab

Travis CI

circleci



Taller

https://aulasoftwarelibre.github.io/taller-de-docker/
https://minikube.sigs.k8s.io/docs/tutorials/kubernetes_101/

Material de apoyo:
https://docs.docker.com/desktop/install/windows-install/

Facultad de
Ingeniería



Universidad de
La Sabana